

Read-Aloud Speaker Script for NDN SF Proposal Defense

Tianxing Ma

PhD Proposal Defense, University of Memphis, June 2026

This document is written as a direct speaking script. Each section corresponds to one slide in the current proposal defense deck.

Contents

1	Slide 1: Title	4
2	Slide 2: What is a service framework?	4
3	Slide 3: Issues with IP-Based Service Frameworks	4
4	Slide 4: Proposal Defense Roadmap	4
5	Slide 5: Why NDN Helps	4
6	Slide 6: NDN Packet Model	5
7	Slide 7: NDN Packet Example: UAV Telemetry and Video	5
8	Slide 8: NDN Data-Centric Security	5
9	Slide 9: NDN Sync and State Vector Sync	5
10	Slide 10: Name-Based Access Control: NAC	5
11	Slide 11: Research Gap	5
12	Slide 12: Related Work: NDN Service Framework Directions	6
13	Slide 13: NDN SF Contribution	6
14	Slide 14: Dissertation Components	6
15	Slide 15: Research Questions	6
16	Slide 16: Proposed Work Status and Timeline	6
17	Slide 17: NDN SF Entity Roles	7
18	Slide 18: NDN SF Invocation Workflow	7
19	Slide 19: NDN SF Access Control: Policy Examples	7
20	Slide 20: Why ABE? Confidential Provider Discovery	7
21	Slide 21: How ABE-Backed Authorization Checks Permission	7
22	Slide 22: Why ABE? Aggregate Permissions, Reuse Identity Credentials	8
23	Slide 23: Runtime Revocation: What Changes and What It Costs	8

24 Slide 24: Request Data: Protected-Discovery Design	8
25 Slide 25: Selective ACK	8
26 Slide 26: Custom Provider Selection Strategy	8
27 Slide 27: Provider Selection Strategies: Built-In Choices	9
28 Slide 28: Provider Selection Strategies: Multi-Provider and Custom	9
29 Slide 29: Completed Foundation: NDNFSF v0.1	9
30 Slide 30: Baselines Before the Comparison	9
31 Slide 31: Network Loss: NDNFSF v0.1 and Baselines	9
32 Slide 32: Historical v0.1: Provider Failures	10
33 Slide 33: Four-Provider Mobility: Experiment Design	10
34 Slide 34: Mobility Results: Controls and Holdout	10
35 Slide 35: NDNFSF v0.1 Evaluation: Availability	10
36 Slide 36: Mobility Evidence: What We Can Claim	10
37 Slide 37: What NDNFSF v0.1 Still Lacks	11
38 Slide 38: UAV Workload	11
39 Slide 39: Requirements of a UAV Network Application	11
40 Slide 40: UAV Requirements → NDNFSF Mechanisms	11
41 Slide 41: Distributed Inference: Splitting a YOLO Model	11
42 Slide 42: Distributed Inference Challenge	12
43 Slide 43: DI Requirements → Proposed NDNFSF-DI Mechanisms	12
44 Slide 44: Post-v0.1: From Application Roles to an Execution Plan	12
45 Slide 45: Proposed: Verify Data Between Assigned Roles	12
46 Slide 46: Proposed NDNFSF-DI Architecture	12
47 Slide 47: Proposed Inference Plan Generation	12
48 Slide 48: Pipeline, Tensor, and Hybrid Tradeoff	13
49 Slide 49: DI Planner Decision	13
50 Slide 50: Preliminary Diagnostic Experiment	13
51 Slide 51: Planned DI Validation	13

52 Slide 52: Planned Evaluation Matrix	13
53 Slide 53: Questions?	13
54 Slide 54: Backup: Risk and Mitigation	13
55 Slide 55: Backup: UAV Validation Details	14
56 Slide 56: Backup: Runtime Metadata Evidence	14
57 Slide 57: Backup: User-Side Admission Control	14
58 Slide 58: Backup: NDNSF Multi-Provider Coordination	14
59 Slide 59: Backup: Why Not ABE Plus Role-Certificate Authorization?	14
60 Slide 60: Backup: Middleware Comparison	14
61 Slide 61: References [1–7]	14
62 Slide 62: References [8–13]	15
63 Slide 63: References [14–21]	15
64 Slide 64: References [22–25]	15

Slide 1: Title

Good morning. My name is Tianxing Ma, and today I will present my dissertation proposal, *NDNSF: A Data-Centric Service Framework for Secure, Collaborative, and Dynamic Networked Applications*. The main contribution is the NDNSF framework itself. During the talk, I will use two application examples: a UAV network application and distributed inference. But I want to be clear from the start. These are not two separate dissertation products. I use them as test cases. They help me find requirements, put pressure on the design, and improve the framework.

Slide 2: What is a service framework?

I want to start with the sentence at the top of the slide. A service framework is a set of application-layer protocols and libraries that turns distributed functions into reusable services. In this talk, reusable means the framework defines how applications discover, authorize, invoke, and compose services. It also hides provider location and implementation details behind service interfaces and policies. Then it manages practical application concerns: provider readiness, selection, retries, security, and data transfer. These are the same concerns we see in UAV control, edge AI inference, storage, sensing, robotics, and many other applications.

Slide 3: Issues with IP-Based Service Frameworks

Existing IP-based service frameworks are useful, so I am not saying they are broken. The slide lists common issues in how the semantics are organized. First, invocation is usually endpoint-oriented: a service call resolves to a reachable host, process, broker, or endpoint abstraction. Second, security is often channel or session centered: TLS protects the communication channel, but stored or cached objects need separate object-level protection. Third, identity, trust, network reachability, and service discovery are usually configured outside the service invocation itself. They are handled by certificates, DNS, registries, load balancers, proxies, or application policy. The bottom bullet is the main point: there is a mismatch between IP's host-centric model and service-oriented applications' data-centric needs. Applications care about named services, authorized data, provider choice, and large data objects, but they often have to bridge hosts, services, trust, and data by hand. NDNSF studies whether NDN names and secured Data can make these pieces part of one data-centric service framework.

Slide 4: Proposal Defense Roadmap

The roadmap has five parts. First is Named Data Networking, or NDN: background and motivation; Reference 1 is the NDN architecture paper. Second is the research gap: what service-layer semantics are missing. Third is the framework: the NDN Service Framework, or NDNSF, including the core design and completed foundation. Fourth is workloads: the UAV network application and distributed inference as validation cases. Fifth is the plan: evaluation matrix, remaining work, risks, and timeline. The blue box gives the central claim: NDNSF is the dissertation contribution; the applications validate and refine the framework.

Slide 5: Why NDN Helps

This slide directly connects the IP-centered problems to the NDN mechanisms that help address them. Endpoint-bound calls become retrieval of named Data from any valid producer or cache. Channel security becomes a verifiable signature attached to each Data packet. Mobility and failover can use another path or producer without changing the requested name. Caching and repository storage are part of the named-data model instead of separate application distribution mechanisms. These properties do not by themselves create a service framework. NDNSF supplies the missing authorization, readiness, invocation, and provider-coordination semantics above NDN.

Slide 6: NDN Packet Model

At the packet level, the slide shows the two basic pieces. An Interest Packet has a Name and optional fields. A Data Packet has a Name, MetaInfo, Content, and Signature. The blue box summarizes the point: Interests name desired Data; Data packets carry names, content, metadata, and signatures. NDN SF builds directly on this. It does not replace the packet model. It defines service requests, ACKs, selections, responses, permissions, and large-object references as named and signed Data objects.

Slide 7: NDN Packet Example: UAV Telemetry and Video

This slide uses UAV telemetry to make the packet model concrete. On the left, the ground station sends an Interest for `/UAV/Drone1/telemetry/status`, and UAV 1 returns Data with position, battery, and health, signed by Drone1. On the right, the same pattern works for a video segment, such as `/UAV/Drone2/video/frame/<segment>`. The takeaway at the bottom is the phrase I want to keep: applications name desired data, and any valid producer, cache, or repository can return signed Data with a matching name.

Slide 8: NDN Data-Centric Security

The slide says security is attached to Data, not to the channel that carried it. A consumer can verify Data after retrieval. Verification still works through caches, repositories, and alternate paths. That is the NDN security base that NDN SF uses. The third bullet also matters: access-control policy is a higher-layer question. So Data validation tells us the packet is authentic and unchanged, but NDN SF still needs service-layer checks for who may invoke a service, which provider was selected, and whether this message is fresh.

Slide 9: NDN Sync and State Vector Sync

NDN is pull-based, so applications need to learn when new Data exists. Alice, Bob, and Ted each publish Data under their own namespace and maintain a local state vector. When they exchange state vectors, Alice can see that Bob has sequence 131 while she only knows 130, and Ted can see that Alice has sequence 113 while he only knows 112. They then fetch the missing named Data by Interest. After those fetches, all participants converge on the same shared dataset without a central broker. References 2 and 3 are the NDN Sync and SVS sources.

Slide 10: Name-Based Access Control: NAC

This slide introduces NAC as background. NAC means Name-based Access Control. The basic idea is that content is encrypted when it is produced, so the Data stays protected even if it is cached, stored in a repository, or forwarded through another path. NAC uses NDN names to express the policy granularity and to name the keys, so encryptors and decryptors can retrieve the needed keys by name. NAC-ABE extends this idea with attribute-based encryption. The UAV example on the right shows why this matters. A UAV can produce encrypted telemetry or video Data, such as a video segment under `/UAV/Drone1/video/seg/42`. That Data can be stored in a cache or repository. A ground station can retrieve the Data by name, but it can decrypt only if it has matching attributes, such as the UAV video namespace and the ground-station role. So the network can move and cache the Data normally, while NAC-ABE controls who can read it.

Slide 11: Research Gap

NDN provides powerful building blocks, including named retrieval, signed Data, caching, access control, computation, and earlier invocation mechanisms. The broader research gap is a secure NDN service framework spanning the service lifecycle: service interfaces, one-to-many Provider discovery and selection, authorization, replay protection, reliability, and large-data delivery. NDN SF v0.1 implements the core

framework functions. The post-v0.1 dissertation work proposes multi-role execution, where Providers are assigned to roles and the Data flowing between those roles is verified.

Slide 12: Related Work: NDN Service Framework Directions

This slide separates four bodies of related work. Computation and composition systems name functions, place computation, or describe dependency graphs. Invocation systems provide named remote-call semantics. NAC, NAC-ABE, Trust Schema, and related work protect content or constrain valid signers. Application-specific systems address aggregation, measurement, or edge security. The gap shown at the bottom is intentionally a targeted collaboration gap, not the complete NDNSF scope. NDNSF v0.1 already covers secure invocation, runtime Provider discovery and selection, authorization, replay protection, and reliable Data delivery. The proposed extension adds multi-role dependency enforcement: after ACK collection, it derives role/dependency rules and admits an intermediate Named Data object only when its name, signer, producer role, dependency edge, and request context match, including when the object comes from an NDN cache.

Slide 13: NDNSF Contribution

The contribution covers the functions introduced in the service-framework definition. The C++ and Python APIs both support generic Request, ACK, Selection, Response, and Targeted invocation without service-specific stubs; trusted same-process local invocation is currently a C++ API. NDNSF also supplies a one-to-many service model, name-based authorization and invocation tokens, and reliable delivery for messages, large objects, and streams. These are implemented foundation capabilities. The proposed extension adds multi-role collaboration: willing Providers are assigned to application roles, allowed Data flow is recorded, and each downstream Provider verifies that its input came from the Provider assigned to an authorized preceding role.

Slide 14: Dissertation Components

This slide is about dissertation components. The top box says NDNSF is the dissertation contribution. The two lower boxes are validation workloads: the UAV requirements workload and the DI requirements workload. I use the UAV workload for mobility, command authorization, freshness, media objects, and missions. I use DI for model files, generated plans, dependency dataflow, and provider collaboration. The feedback arrow is important: these workloads do not just consume the framework; they feed requirements back into the framework design. So I am not claiming that the UAV application or the inference planner is the final product. They are there to test whether NDNSF provides the service mechanisms these systems need.

Slide 15: Research Questions

RQ1 asks which service-layer functions are required for secure invocation, dynamic multi-Provider selection, authorization, and reliable Data delivery over NDN. RQ2 asks how NDNSF can assign Providers to application roles and ensure that a downstream Provider accepts Data only from the Provider assigned to an authorized upstream role, including cached Data. RQ3 studies the reliability, security, and latency tradeoffs of publication, selection, Targeted invocation, and bounded response retry or Provider reselection. RQ4 applies the framework to UAV mobility and control. RQ5 applies it to reusing model files, assigning model parts to available devices at runtime, and running one model across multiple devices.

Slide 16: Proposed Work Status and Timeline

This table distinguishes completed mechanisms from remaining evidence and administrative milestones. The completed framework includes Targeted invocation, trusted C++ local calls, large-data support, and an execution-plan schema. August 2026 is reserved for Plan-to-Data enforcement over both direct

and cached Data and for measuring its security and latency overhead. September 2026 covers UAV role and dependency validation, and October 2026 covers real-model distributed-inference and failure gates. In November and December 2026, I will freeze the evidence and figures, integrate the dissertation, and submit a complete draft to the chair. Committee review runs from January through March 2027, with the defense planned no later than mid-February and ProQuest submission targeted for March 5, ahead of the March 26 deadline.

Slide 17: NDNSF Entity Roles

This slide shows the three NDNSF entity roles. The Service Controller is the startup authority. At startup, the Service User and Service Providers obtain the permissions, keys, and certificates they need for authorized service participation. During invocation, the Service User coordinates directly with one or more Service Providers. The blue box says the main point: startup material comes from the controller, but Request, ACK, Selection, and Response happen directly among the runtime entities through named Data messages. That role split keeps the runtime distributed while still giving users and providers the security material they need.

Slide 18: NDNSF Invocation Workflow

The invocation workflow has four logical messages: Request, ACK, Selection, and Response. The left box explains the underlying Sync mechanism. First, the sender publishes a logical message as a signed, named Data object. Second, Sync announces the new Data name. Third, the receiver sends an Interest for that name and obtains the Data. The right box explains what the application sees through the SVS Pub/Sub API. A publish call stores the Data and announces its name; a subscription learns the name, fetches the Data, and invokes the callback. Thus, Pub/Sub hides the two-step announce-then-fetch delivery from NDNSF. Optional piggybacking can carry an already signed Data packet with Sync and avoid the separate Interest/Data exchange; it is an optimization, not a change to the logical message workflow. Provider readiness is reported in ACK, provider choice is expressed in Selection, and final execution produces the Response.

Slide 19: NDNSF Access Control: Policy Examples

These examples distinguish two policy directions. A Provider policy defines which named services an identity may offer. A User policy defines which Provider/service namespaces that User may invoke. The three User entries on the slide correspond to services actually offered by Drone 1 or Drone 3. Neither policy is written for one fixed host, socket, or channel, so the authorization remains name-based as Providers move or change.

Slide 20: Why ABE? Confidential Provider Discovery

Providers need enough information to decide whether to participate before the User selects one. The discovery descriptor can contain an observation area, deadline, sensor requirement, or model requirement. ABE makes this content readable by Providers authorized for the service without listing their identities. It does not hide the descriptor from authorized but unselected Providers, and does not hide outer Data names or traffic patterns. In the request-scoped confidentiality path, application input and its key material are restricted to selected Providers through Selection. A Trust Schema alone does not encrypt content, but role authorization can be paired with ABE or group encryption.

Slide 21: How ABE-Backed Authorization Checks Permission

The Requester Challenge is named UserToken in the implementation. A Provider demonstrates access to the service-protected Request by returning this challenge in ACK. ACK also contains a Provider Challenge protected for Users authorized to invoke the service; Selection returns the chosen Provider's challenge.

Each side must still validate signatures, participant identities, the service, the active request, and one-time state. These checks demonstrate access to decryption capability for this exchange, not physical resources or correct task execution. Tokens alone do not establish all authorization or replay guarantees.

Slide 22: Why ABE? Aggregate Permissions, Reuse Identity Credentials

DNMP uses role signing keys and a Trust Schema to constrain commands. A role may cover several permissions, so we do not assume one certificate per service in DNMP. Our per-service role-certificate alternative would issue an entity a signing credential for each authorized service role. NDNFSF instead aggregates that entity's service permissions in an individually issued KP-ABE decryption key within one authority and parameter generation, while reusing its identity-signing credentials. This avoids the additional per-service signing credentials in that particular alternative. DKEY distribution, updates, size, and cryptographic processing remain costs; fewer objects do not prove lower total cost.

Slide 23: Runtime Revocation: What Changes and What It Costs

NDNFSF participants retrieve and validate Controller-signed authorization status; runtime checks then reject revoked invocations. ABE generation rotation and refreshed DKEYs protect later exchanges encrypted under the new generation. Current rotation is Controller-wide, so unaffected authorized members may also need DKEY updates. DNMP discusses rotating encryption keys to restrict future measurement-data access, but does not detail a comparable online command-authorization revocation protocol. For a fair certificate-based alternative, we propose revoking the relevant role credential and checking signed revocation status at execution nodes. That is a comparison design, not a claimed DNMP implementation. Both designs depend on update propagation and cannot erase keys or plaintext already disclosed. NDNFSF has focused integration-test evidence, not a measured proof of cheaper or instantaneous network-wide revocation.

Slide 24: Request Data: Protected-Discovery Design

The protected-discovery design separates the discovery descriptor from the full application input. Request Data names the service and request, carries the descriptor used for the positive-ACK decision, and includes the Requester Challenge. In the request-scoped confidentiality path, Selection restricts the application input and key material to selected Providers. This is the revised protection scope, not a claim that every historical v0.1 Request path already separated the two. The named Data representation remains the same; signature, authorization, and active-request checks apply after retrieval.

Slide 25: Selective ACK

Selective ACK is about the status carried by the ACK, not about omitting the ACK. In the normal discovery path, every Provider returns a signed ACK. A positive ACK means the Provider is willing and currently capable; a negative ACK means it received the request but is not ready or cannot serve it now. ACK metadata can report load, wait time, location, model availability, or safety state. The User selects only among positive ACKs. Explicit overload suppression and the Targeted fast path are separate exceptions, not the normal Request-ACK-Selection workflow.

Slide 26: Custom Provider Selection Strategy

The key sentence on this slide is that provider selection is user-side, policy-driven, and performed after ACK collection. The User collects all ACKs during an ACK window, while only positive ACKs become selectable candidates; negative ACKs remain readiness evidence but are not selected. Built-in choices include first responding, random, and all selected. A custom policy can use latency, load, capability, location, or safety metadata. So the framework keeps ACK collection common, but lets the user-side policy decide how to choose among positive candidates.

Slide 27: Provider Selection Strategies: Built-In Choices

This slide shows the two simple built-in provider selection choices. Both start from the same Request and ACK collection step. In first responding, the user selects the first ready provider that returns a valid ACK. This is useful when the goal is a simple low-latency choice. In random selection, the user picks randomly among valid ACK candidates. This can spread work across providers without writing a special policy. The key point is that even these simple choices are framework-level provider selection, not application glue code.

Slide 28: Provider Selection Strategies: Multi-Provider and Custom

This slide shows the two more flexible provider selection choices. All responders means the user can select every ready provider that returned a valid ACK. That supports multi-provider execution, where more than one provider may work on the service. Custom selection means the application provides its own policy after ACK collection. The policy can use latency, load, capability, location, safety state, or other ACK metadata. So the framework keeps Request and ACK collection common, but lets the user-side policy choose one provider, several providers, or all ready providers.

Slide 29: Completed Foundation: NDN SF v0.1

This slide marks the exact historical boundary. The reproducible May 20 repository baseline is tag v0.1, commit `be9421f...`, tree `c9cc0c...`. It includes the generic dynamic C++ API shown on the right, multi-Provider ACK collection, selection strategies, the NDN-SVS patch, and the original experiments. It does not include Python bindings or role-based collaboration planning. Before introducing the extension, I will show exactly what the completed v0.1 evidence establishes.

Slide 30: Baselines Before the Comparison

The systems retain different semantics. A gRPC resolver can supply several backend addresses to one channel, and a load-balancing policy selects one ready backend for each RPC; this is not the same as sending one request to every Provider. The network-loss gRPC-1 baseline uses one configured endpoint. The later mobility gRPC-SEQ-4 baseline preregisters four Provider endpoints and retries them sequentially in application logic rather than using native gRPC load balancing. Retry is also separate from load balancing and normally requires an explicit retry policy beyond narrowly scoped transparent retries. NSC binds a named call to one executor, while NDN SF uses its Request-ACK-Selection-Response workflow to observe multiple responding Providers before selection. The network-loss numbers are a configuration-aligned contextual comparison, not one statistically matched campaign: NDN SF uses the submitted v0.1 measurements, while NSC and gRPC each use one new run per loss under the displayed topology, workload, five-second deadline, and zero application retries. Security is also not equivalent: the gRPC baseline is insecure, NSC skips signature verification, and the MiniNDN build uses a dummy keychain.

Slide 31: Network Loss: NDN SF v0.1 and Baselines

This slide restores the original NDN SF v0.1 network-loss measurements and places new single-run NSC and gRPC baselines beside them. The path is Memphis-CSU-UCLA, the offered load is ten requests per second, the service delay is five milliseconds, and the baseline deadline is five seconds with no application retry. NDN SF reports 599 of 600, 598 of 600, and 539 of 600 completions at one, three, and ten percent link loss; the paper describes the ten-percent value as the best measured result and attributes the resilience to SVS retry. In the new runs, NSC completes 541, 444, and 208 of 600 requests, or 90.17, 74.00, and 34.67 percent. gRPC completes 600, 600, and 541 of 600, or 100.00, 100.00, and 90.17 percent. These n-equals-one baselines provide configuration-aligned context only; there is no variance or significance claim, and the mixed batches are not a statistically matched campaign.

Slide 32: Historical v0.1: Provider Failures

This historical topology places the User at Memphis and identical service Providers at UCLA, WUSTL, and UIUC. Every ten seconds, each Provider independently enters a ten-second rejection period with probability ten, twenty, or thirty percent. An unavailable NDNFSF Provider suppresses its ACK, allowing FirstResponding to choose another valid runtime candidate. Under these evaluated v0.1 configurations, NDNFSF stays above 96 percent success at the thirty-percent setting. NSC and gRPC each use one configured executor or endpoint and do not retry another target, so this result is motivation rather than the confirmatory four-target comparison on Slides 32 and 33.

Slide 33: Four-Provider Mobility: Experiment Design

This slide separates two matched analysis layers on the same one-AP, four-Provider topology. The all-request controls use 50, 100, and 150 meter radii to show how physical coverage limits unconditional success. The independent switching test uses the 100-meter condition. Each client records the actual request-publication time, and the analyzer reconstructs the coverage state at that instant. A request enters the conditional comparison only when the initial Provider is unavailable but another Provider is reachable. The 4.05-second request phase places publications between 100-millisecond gate updates. gRPC-SEQ-4 and NSC-4 try four preregistered targets sequentially, while NDNFSF discovers ACK candidates at runtime and uses FirstResponding with bounded response reselection. All systems use byte-identical traces, five requests per second for 60 seconds, a one-second attempt or ACK timeout, a five-second global deadline, and disabled admission and proactive health routing.

Slide 34: Mobility Results: Controls and Holdout

The left panel is intentionally simple. As the AP radius grows from 50 to 100 to 150 meters, all three systems move from roughly 54–55 percent success to roughly 98 percent and then 100 percent. The main driver is which Providers are actually covered when a request is published, not a universal advantage for one system. The right panel focuses only on 1,312 matched requests where the initial Provider was unavailable but another Provider was reachable. NDNFSF's tail latency is 597 milliseconds lower than gRPC and 2,618 milliseconds lower than NSC; the error bars show the 95-percent confidence ranges. Success is 99.92 percent for NDNFSF, 98.86 percent for gRPC, and 98.78 percent for NSC, but no overall success-rate advantage is claimed because actual Provider coverage at publication time dominates success. The supported claim is therefore lower tail latency when switching is required, with comparable success, not universal mobility superiority.

Slide 35: NDNFSF v0.1 Evaluation: Availability

This real-machine test uses an Odroid User with laptop and PC Providers. The laptop is unavailable during the first half, while the PC is unavailable during the middle half. Their unavailable windows overlap for one quarter of the run, so at least one Provider is available for only 75 percent of the total time. That 75 percent is a provider-time coverage ceiling, not a guaranteed request-success rate. NDNFSF reaches 74.62 percent at 10 RPS and 73.73 percent at 30 RPS. The small gap is expected because failover is not instantaneous: requests already in flight may expire while the User collects ACKs, performs selection, handles a timeout, and retries with another Provider. SVS/Sync retransmission can recover lost named Data, but it cannot make an unavailable Provider answer; if recovery completes after the deadline, the request is still counted as failed. The experiment therefore shows that NDNFSF follows changing Provider availability instead of binding the User to one endpoint.

Slide 36: Mobility Evidence: What We Can Claim

This slide separates three evidence levels. The original v0.1 failure and availability demonstrations motivate runtime Provider discovery but use single-target NSC and gRPC configurations. The matched

all-request controls use identical traces and four targets; they show that physical coverage dominates unconditional success and do not support a universal NDN v0.1 advantage. The independent switching test then isolates requests for which the initial baseline target is unavailable while another Provider is reachable. It supports a narrower conclusion: NDN reduces tail latency relative to sequential retry over pre-registered targets when switching is actually required, with comparable success. Architectural endpoint discovery and the measured conditional latency result should remain distinct claims.

Slide 37: What NDN v0.1 Still Lacks

The completed results establish runtime discovery, service ACKs reporting willingness and status, Provider selection, request-level authorization, and resilience. They do not yet assign different Providers to different application roles, record which role may send Data to a downstream role, or verify that a received object came from the Provider assigned to an authorized preceding role. That verification must make the same decision whether the Data arrives directly or is retrieved from an NDN cache. UAV and distributed inference make these missing functions concrete.

Slide 38: UAV Workload

The v0.1 results showed dynamic selection. UAV now asks whether that foundation can support mobile, command-sensitive, media-heavy, and role-dependent workflows. The ground station can invoke drone services, but the drones can also invoke one another. A drone may therefore act as both an NDN User and Provider during a mission. UAV-APP is a requirements workload, not a claim that this dissertation will build a complete UAV product.

Slide 39: Requirements of a UAV Network Application

These five items define the UAV scope: mobility, command authorization, telemetry freshness, video and recording objects, and multi-drone missions. The dissertation does not claim a comprehensive UAV application; it develops the framework mechanisms needed to support this class of application. The next slide maps every requirement to a concrete mechanism and evaluation target.

Slide 40: UAV Requirements → NDN Mechanisms

For generic one-to-many calls, Providers return service ACKs reporting willingness and runtime status, and NDN selects among responding Providers. Bounded response reselection is optional and is not the default behavior. Targeted invocation skips ACK and Selection only after the Provider and service have completed token bootstrap. Command authorization uses NAC-ABE, one-time tokens, and replay checks; telemetry uses freshness and lifecycle state; media uses immutable objects and segmented retrieval. For a multi-drone mission, NDN assigns Providers to roles and verifies that each downstream Provider accepts Data only from the Provider assigned to an authorized preceding role.

Slide 41: Distributed Inference: Splitting a YOLO Model

This YOLO example introduces the two kinds of model split before the NDN-DI architecture. Following Figure 1, a horizontal cut means a layer-wise cut along the model's layer sequence: contiguous layer ranges become pipeline stages such as Backbone, Neck, and Detection Head, and each stage passes named intermediate activations to the next. A vertical cut means a tensor-wise cut within a layer or stage: each device computes a partial view or slice of the same computation, and partial outputs may require exchange or aggregation. The labels describe the model dimension being partitioned, not the geometric orientation of the line drawn in Figure 1. I use the word partition rather than rank because these are pieces of one computation, not an ordering. A hybrid layout may combine both kinds of split.

Slide 42: Distributed Inference Challenge

The example collaboration plan makes the DI problem concrete. A YOLO request needs signed NDN Data objects containing reusable layer artifacts, ONNX Runtime plus an NDN-SF-DI adapter that interprets and splits the model, Provider offers describing devices and resources, an input image, and a deadline. After collecting the offers, the planner may assign the Backbone role to P2's CPU, the Neck role to P3's GPU, and the Detection Head role to P4's GPU. A pipeline stage is one kind of DI role. The proposed plan states which Provider runs each role, on which device, with which model part, and which named intermediate Data may flow downstream.

Slide 43: DI Requirements → Proposed NDN-SF-DI Mechanisms

The title marks these as proposed NDN-SF-DI mechanisms rather than completed V3 evidence. Distributed inference must run models that do not fit one device, use heterogeneous CPU and GPU Providers, reuse signed layer artifacts rather than copy a full model for every request, send intermediate tensors only along authorized role edges, and recover when a selected Provider becomes unavailable. NDN-SF supplies the service messages and authenticated Provider choice; NDN-SF-DI proposes the model interpretation, split, device assignment, and bounded replanning or Provider reselection.

Slide 44: Post-v0.1: From Application Roles to an Execution Plan

Now that both UAV and YOLO examples have been introduced, this slide generalizes their common planning process in terms of roles. The application describes its roles, services, allowed Data flow, and constraints. One Request elicits signed ACKs from willing Providers; those ACKs report capabilities, load, and resources. An application planner assigns Providers to roles, and NDN-SF checks that every required role has an eligible Provider. The UAV example assigns the Capture, Detection, and Mapping roles and records the allowed Data flow between them.

Slide 45: Proposed: Verify Data Between Assigned Roles

A plan is useful only if it constrains execution. In this YOLO example, P2 is assigned the Backbone role and P4 the Detection Head role. Before P4 processes the feature tensor, it checks that the object belongs to the current request, that P2 signed it, that P2 is assigned to the authorized preceding Backbone role, and that the Data name is allowed. The same valid object is accepted from a cache; an unassigned P3, an undeclared name, or a replayed old tensor is rejected. This is the proposed Plan-to-Data mechanism.

Slide 46: Proposed NDN-SF-DI Architecture

The title and diagram explicitly mark this as the proposed NDN-SF-DI architecture, avoiding confusion with Targeted invocation. A model request collects signed Provider offers, the DI planner chooses the model split and placement, and each selected Provider receives instructions containing its role, model part, and device. Providers prepare only their assigned model parts from reusable signed layer artifacts in a Repo or cache. Intermediate tensors connect the assigned roles. An ACK reports availability but does not reserve a GPU, so every Provider rechecks the selected device immediately before loading.

Slide 47: Proposed Inference Plan Generation

Plan generation combines the model graph and reusable signed layer-artifact catalog with the immutable set of signed Provider offers. Each offer describes the Provider's available devices, capacity, and cached artifacts. The planner chooses the split, Providers, and devices, checks that the plan fits, and emits per-Provider role instructions. In the example, stage zero runs on P2's CPU while two partitions of stage one use P4's two GPUs.

Slide 48: Pipeline, Tensor, and Hybrid Tradeoff

Layer-wise pipeline partitioning assigns contiguous layer ranges and transfers intermediate tensors between stages. Tensor-wise partitioning divides selected layers or stages across devices but adds exchange and redistribution cost. The planner chooses a hybrid layout using model compatibility, reuse, per-device feasibility, topology, intermediate-data size, and network cost.

Slide 49: DI Planner Decision

Planning begins only after the immutable ACK set closes. Decision evidence includes model and device fit, intermediate-data exchange, network topology, Provider offers, and reusable signed layer artifacts. Capacity is checked separately for every selected device; incompatible memory is never summed. A reuse-first strategy may prefer already cached signed artifacts only after all model, resource, and security requirements hold.

Slide 50: Preliminary Diagnostic Experiment

The figure now makes the measured placement explicit. In the shared-backbone layout, the Backbone runs at UCLA, Head A at Arizona, Head B at WUSTL, and Merge at NEU. In the single-Provider control, all four roles run at UCLA. The same MiniNDN harness and small ONNX NativeTracer workload were used for both layouts, with ten successful real-ONNX runs per layout. Single-Provider execution was faster because fixed cross-Provider intermediate-data exchange dominated. This diagnostic does not validate the proposed V3 reuse, hybrid partitioning, multi-GPU placement, or final device recheck.

Slide 51: Planned DI Validation

Slide 48 is earlier harness evidence; this slide is the V3 validation contract. The gates cover model-file identity and reuse, CPU and zero, one, or multiple accelerator offers, no ACK-time reservation, final resource recheck, pipeline and multi-device execution, direct/cache security equivalence, and integration-to-MiniNDN-to-sealed-container qualification. Until these gates close, the proposal does not describe V3 as validated.

Slide 52: Planned Evaluation Matrix

Do not read this table as completed results. RQ1 uses source-backed gap analysis; RQ2 and RQ3 test the core plan, admission, reliability, security, and latency properties; RQ4 uses MiniNDN, PX4 SITL, and selected machines; RQ5 uses integration tests, MiniNDN, and sealed real-model/container qualification. Only the preliminary small-model campaign on Slide 47 is complete within the DI row.

Slide 53: Questions?

The dissertation thesis is that NDNSF can turn ACK-derived roles and named dependencies into enforceable policy for direct and cached Named Data. The v0.1 implementation and initial evidence exist; Plan-to-Data enforcement and its full validation remain proposed. Thank you. I am happy to take questions.

Slide 54: Backup: Risk and Mitigation

The main risks are scope, latency, large-data overhead, and variable real-machine tests. UAV and DI remain validation workloads rather than full products. Protocol phases and Data transfer are measured separately. MiniNDN and SITL provide repeatable baselines, while real machines are used selectively. The revised scope also requires that Plan-to-Data claims remain proposed until enforcement and negative-test evidence are complete.

Slide 55: Backup: UAV Validation Details

UAV validation covers the Targeted command lifecycle, typed state freshness, large signed media objects, and mission recovery. Multi-drone scenarios will additionally test role assignment and named dependency admission. The goal is to determine whether generic NDNF mechanisms support mobile, command-sensitive collaboration, not to claim a complete flight application.

Slide 56: Backup: Runtime Metadata Evidence

These results show that runtime metadata affects behavior rather than serving as descriptive text. Adaptive admission changes whether a request enters the pipeline under load. Positive and negative ACK status exposes Provider readiness before selection while still returning a signed ACK for the normal discovery path. In the newer design, validated ACK metadata also becomes the input to collaboration planning.

Slide 57: Backup: User-Side Admission Control

User-side admission bounds outstanding requests before they enter the Request-ACK-Selection-Response pipeline. A window is updated from observed latency, timeouts, and queue pressure, so requests may be admitted, queued, or rejected. This is overload control at the requester and should not be confused with the proposed Provider-local Data-admission rule for collaboration dependencies.

Slide 58: Backup: NDNF Multi-Provider Coordination

The May 20 framework already treated Provider coordination as a framework concern: one Request could reach multiple authorized Providers, Providers returned service ACKs reporting willingness and runtime status, and the user selected one or more responders. That completed behavior is multi-Provider discovery and selection, not yet role-based collaboration. The newer design adds explicit roles and named dependency edges after ACK collection, and the proposed enforcement step turns those assignments into Provider-local Data-plane rules.

Slide 59: Backup: Why Not ABE Plus Role-Certificate Authorization?

ABE for confidential discovery combined with role-certificate execution authorization is a valid alternative. The reason for retaining ABE-backed challenges is to reuse attribute-based service permission management rather than introduce an additional service-role signing-credential structure. This is not free: Provider DKEYs are needed for protected discovery, while User permission DKEYs and challenge processing are extra authorization costs. A fair comparison holds permissions, confidentiality, replay behavior, and revocation requirements constant, then counts credential issuance, updates, bytes, and processing. We do not claim that a Trust Schema cannot authorize execution or that ABE is universally cheaper.

Slide 60: Backup: Middleware Comparison

gRPC and ROS 2 are useful systems with different runtime abstractions, as References 17 and 18 document. gRPC provides endpoint-oriented RPC together with name resolution, load balancing, and retry facilities. ROS 2 provides a distributed robotics graph. NDN instead makes named, signed, cacheable Data the network object. NDNF studies the additional service semantics needed above that object model, including ACK-based discovery and, in the newer design, role-and-dependency collaboration. It is not presented as a universal replacement for either middleware.

Slide 61: References [1–7]

This lookup slide contains the foundational references for NDN, Sync, SVS, NAC, NAC-ABE, NFN, and NFaaS. Their numbers are placed at the first relevant mentions in the main presentation. I would use this slide only to answer a committee question.

Slide 62: References [8–13]

This lookup slide covers CFEC, RICE, NSC, MIA-NDN, DNMP, and SECaaS. These references support the comparison of placement, invocation, measurement, and application-specific service frameworks. The main talk uses the reference numbers beside the corresponding claims.

Slide 63: References [14–21]

This lookup slide covers Compute First Networking, CaSCON, Trust Schema, gRPC, ROS 2, enhanced NAC-ABE, access-controlled in-network processing, and data-centric service composition. These sources sharpen the novelty boundary: the proposed contribution is not named computation, caching, content authentication, or graph planning alone, but the compilation of an ACK-derived collaboration plan into Provider-local admission rules for direct or cached Named Data.

Slide 64: References [22–25]

This lookup slide provides the tool and protocol references placed at the first mentions of Mini-NDN, PX4 SITL, NDN Repo, and ONNX. They document the emulation, flight simulation, repository protocol, and model representation used by the proposed validation workloads.